

Helm pro Kubernetes (balení a nasazování aplikací)

Cíl dokumentu

Tento dokument popisuje Helm z pohledu týmu, který:

- nasazuje aplikace do Kubernetes opakovaně ve více prostředích,
- potřebuje parametrizovatelné šablony manifestů,
- chce sdílet balené aplikace formou chartů.

1. Co je Helm

Helm je balíčkovací nástroj pro Kubernetes:

- **chart** = balíček manifestů a metadat,
- **release** = konkrétní nasazení chartu v clusteru,
- Helm CLI umožňuje instalovat (`install`), upgradovat (`upgrade`), rollbackovat (`rollback`) a mazat (`uninstall`) release.

Helm přenáší koncepty známé z Linux balíčků (`apt/yum`) do světa Kubernetes.

2. Struktura chartu

Typická struktura:

- `Chart.yaml` – metadata chartu (název, verze, závislosti),
- `values.yaml` – default hodnoty pro parametry,
- `templates/` – YAML manifesty šablonované pomocí Go templating,
- `charts/` – závislé subcharty,
- `.helmignore` – soubory ignorované při balení.

Důležité soubory v `templates/`:

- `deployment.yaml` – definice Deploymentu,
- `service.yaml` – definice Service,
- `ingress.yaml` – definice Ingress,
- `_helpers.tpl` – pomocné template (např. generování názvů),
- `NOTES.txt` – text, který se zobrazí po instalaci release.

3. Values a jejich přepis

`values.yaml` obsahuje výchozí konfiguraci chartu.

Hodnoty se přepisují v pořadí:

1. `values.yaml` v chartu,
2. `values.yaml` v parent chartu (u subchartu),
3. hodnoty z `-f / --values` souborů při instalaci/upgrade,
4. hodnoty z `--set` parametrů.

Příklad použití:

- `helm install myapp ./chart --values=values-prod.yaml,`
- `helm upgrade myapp ./chart --set image.tag=1.2.3.`

Uvnitř template přistupuješ k hodnotám přes objekt `.Values`:

- `{{ .Values.image.repository }}`,
- `{{ .Values.service.type }}`.

4. Go templating v Helm

Šablony v `templates/` jsou Go templates rozšířené o Helm funkce.

Základní objekty:

- `.Release` – informace o release (jméno, namespace, revize),
- `.Chart` – metadata chartu,
- `.Values` – hodnoty z `values`,
- `.Capabilities` – verze Kubernetes API,
- `.Files` – přístup k souborům v chartu.

Časté konstrukce:

- podmínky: `{{ if .Values.ingress.enabled }} ... {{ end }}`,
- cykly: `{{ range .Values.extraEnvs }} ... {{ end }}`,
- funkce: `{{ default "ClusterIP" .Values.service.type }}`.

5. Subcharty a global values

Chart může mít závislosti (subcharty):

- definované v `Chart.yaml` → `dependencies`,
- například databáze, ingress controller, cache.

Konfigurace subchartu se dělá přes prefix v `values.yaml`:

- `postgresql.enabled: true,`

- `postgresql.primary.resources: ....`

global values umožňují sdílenou konfiguraci mezi chartem a subcharty:

- `global.imageRegistry,`
- `global.labels.`

6. Versioning a repozitáře

Charty se verzují podobně jako Docker imagy:

- verze chartu (`version`) v `Chart.yaml` – semver,
- verze aplikace (`appVersion`) – informativní.

Charty se publikují do Helm repozitářů:

- veřejné (Artifact Hub, Bitnami, atd.),
- privátní (S3, GCS, GitLab Pages, Nexus, atd.).

Instalace z repozitáře:

- `helm repo add bitnami https://charts.bitnami.com/bitnami,`
- `helm search repo bitnami/postgresql,`
- `helm install db bitnami/postgresql.`

7. Helm lifecycle

Základní příkazy:

- `helm install` – vytvoří nový release,
- `helm upgrade` – aktualizuje release na novou verzi chartu/values,
- `helm rollback` – vrátí release na starší revizi,
- `helm uninstall` – smaže release (volitelně ponechá persistentní data),
- `helm list` – vylistuje release v namespace.

Helm uchovává historii release (vytváří `ConfigMap/Secret` v namespace),

- to umožňuje `rollback`.

8. Debugging chartů

Užitečné příkazy:

- `helm template ./chart` – vyrenderuje manifesty na stdout, ale nic nenasadí,
- `helm install --dry-run --debug` – simuluje instalaci a ukáže detaily,
- `helm get manifest <release>` – zobrazí manifesty nasazeného release.

Při problémech:

- ověř, jak vypadají vyrenderované manifesty,

- ověř Kubernetes objekty (`kubectl describe pod`, `kubectl get events`),
- sleduj logy aplikace a controllerů.

9. Helm a prostředí

Typický pattern:

- jeden chart pro aplikaci,
- různé values soubory pro prostředí (`values-dev.yaml`, `values-prod.yaml`),
- CI/CD pipeline, která vybere správný soubor a případně doplní dynamické hodnoty (`--set image.tag=...`).

Pro složitější scénáře se používají nadstavby (Helmfile, ArgoCD ApplicationSet, Flux Kustomization + HelmRelease).

10. Helm v kombinaci s Terraformem

Terraform s providerem `helm` umí:

- spravovat Helm release jako součást IaC,
- zajistit pořadí – např. nejdřív vytvořit cluster, pak nasadit Ingress Controller, potom aplikace.

Výhody:

- konsolidace infrastruktury a aplikací v jednom Terraform projektu,
- audit a historie změn ve verzovacím systému.

Důležité je sladit:

- verze helm chartů,
- secrets a config (předávání dat z Terraformu do Helm values).